SILENT · 2인 협동 방탈출 기술문서

두 사람의 탈출을 완성하는 클라이언트·서버 설계

시작 화면 → 공동 준비 → 탐색과 정답 → 방 진행 → 엔딩과 종료

플레이어가 보고 누르는 장면에서 출발해 데이터, 명령, 상태와 결과 이벤트를 따라갑니다. 큰 기능에서 작은 기능으로 내려가며, 같은 순간 클라이언트와 서버가 각각 무엇을 책임지는지 설명합니다.



그림 1 · 시작 화면에 사용되는 프로젝트 배경 원본. 실행 화면이나 온라인 성공을 증명하는 캡처는 아닙니다.

> 읽는 순서: 플레이 경험 → 데이터·명령의 흐름 → 표현 결과 → 구현과 설계 의도. 관계도·요청 순서도·데이터 연결도를 함께 제공합니다.

작성 기준: 2026-10-26 현재 로컬 코드·CSV·설정 및 기존 기술 문서.
범위: Unity 클라이언트와 Node.js 게임 서버의 통합 구조. 공개용 요약본.

목차

플레이 흐름을 먼저 읽고, 뒤에서 공통 설계·자동화·검증 근거를 확인합니다.

0. 전체 플레이 흐름과 기술 선택	**3**
1. 클라이언트·게임 서버·음성 관계	**4**
2. 시작 화면에서 탈출하기를 누른다	**5**
3. 두 사람의 준비가 끝나면 시작한다	**6**
4. 내려받은 데이터가 게임 정의가 된다	**7**
5. 역할에 맞는 방과 사물을 만든다	**8**
6. 플레이어가 이동하고 손이 움직인다	**9**
7. 사물을 바라보고 확대 화면을 연다	**10**
8. 정답을 제출하고 서버가 판정한다	**11**
9. 문을 열고 다음 방으로 이동한다	**12**
10. 이동에 맞춰 공간의 소리가 바뀐다	**13**
11. 다른 방의 상대와 음성으로 협력한다	**14**
12. 미리 계산한 조명으로 공간을 표현한다	**15**
13. 마지막 방은 두 사람을 기다린다	**16**
14. 영상이 끝나면 결과와 종료를 보여준다	**17**
15. 퇴장과 파괴까지 객체 수명을 닫는다	**18**
16. 이 흐름을 지탱하는 계층과 패턴	**19**
17. 통신 계약·주기·실패를 다루는 경계	**20**
18. 제작·검증·배포·로그의 자동화	**21**
19. 최적화와 현재 확인 범위	**22**
20. 코드와 기존 문서 찾아보기	**23**
21. 공개 범위와 저장소	**24**

> 공개용 읽기 기준: 클래스명은 실제 코드 식별자를 사용합니다. 서비스 주소·토큰·정답 원문은 싣지 않습니다. 코드 구현과 실제 다중 PC 검증은 구분합니다.

0. 전체 플레이 흐름과 기술 선택

처음 보이는 것은 시작 화면이지만, 두 사람이 함께 진행하려면 화면 뒤의 준비와 판정 순서가 먼저 맞아야 합니다.

0.1 한 번의 플레이가 완성되는 경로

플레이 단계	클라이언트의 일	서버가 확정하는 일
시작·입장	탈출하기 입력·설정 선택	자리·역할 A/B·참가 토큰 발급
대기·준비	데이터 검증·방·플레이어 생성	두 참가자의 준비 완료
탐색·협력	이동·시점·사물 확대·음성	세션의 유효성과 생존
정답·방 진행	입력 전송·결과 표시·문 회전	정답·문 허가·누적 방 해금
공동 완료·종료	영상·탈출 시간·결과 화면	마지막 방 양쪽 정답 조건

0.2 사용 언어와 이 구성에서의 선택 이유

클라이언트는 C#과 Unity 6000.2.8f1을 사용합니다. 이동·충돌·화면·영상처럼 엔진과 가까운 작업은 컴포넌트로 연결하되, 판정·시간·입력 상태는 가능한 한 일반 C# 객체로 분리합니다. 유니티 장면 없이도 규칙을 검사하기 위한 경계입니다.

서버는 Node.js 24 이상과 TypeScript를 사용합니다. HTTP 요청과 비동기 입출력을 처리하면서 타입 계약으로 세션 결과·CSV 정의를 표현합니다. 현재 서버는 단일 프로세스의 메모리 세션 하나를 소유하므로, 복잡한 분산 저장보다 작은 책임 경계와 추적 가능한 판정 흐름에 맞춘 구성입니다.

0.3 왜 HTTP와 Protobuf인가

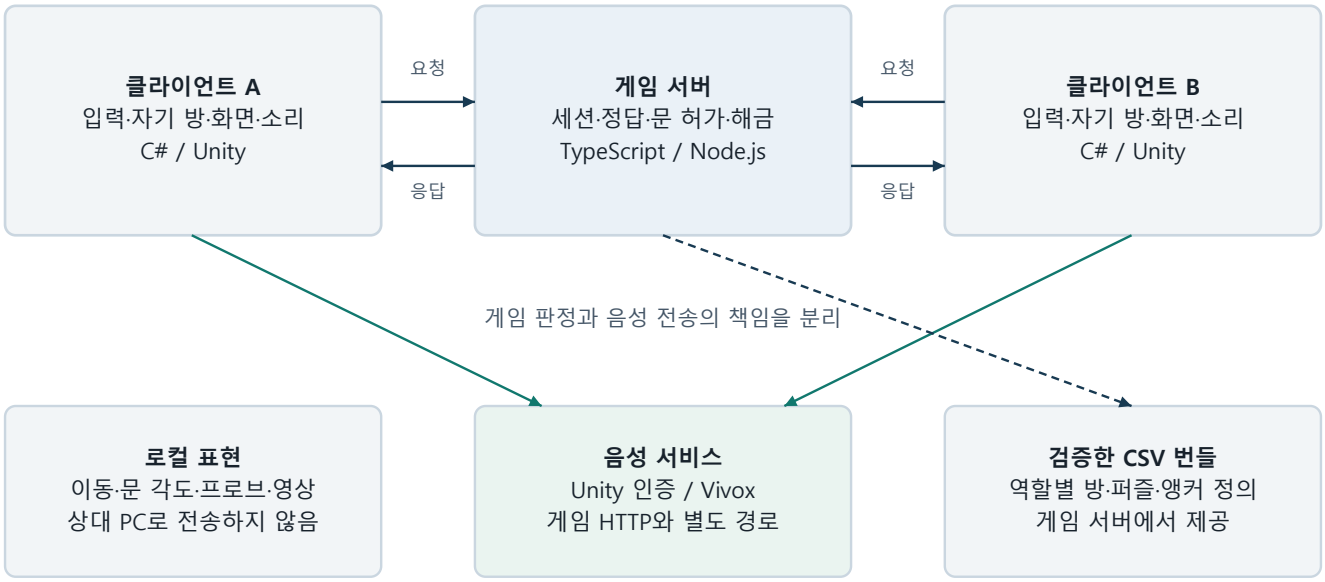
정답 제출·준비 보고·문 확인은 연속 위치 스트림보다 요청과 응답으로 설명하기 쉬운 작업입니다. HTTP는 호출 경로와 실패 상태를 명시하고, Protobuf는 두 언어가 같은 메시지 정의를 사용하게 합니다. 위치·물리·문 각도의 고주파 동기화는 현재 범위가 아니므로 이 방식이 적합합니다.

> 설계의 핵심: 클라이언트는 의도를 요청하고 서버는 진행을 판정합니다. 화면을 바꾸는 일과 진행 권한을 확정하는 일을 분리합니다. Protobuf를 쓰지만 gRPC 서버는 아닙니다.

구현 연결: 양쪽 README · src/main.ts · src/models/GameSession.ts · World/GameFlowBootstrapper.cs

1. 클라이언트·게임 서버·음성 관계

같은 세션에 속해도 각 PC는 자기 방과 자기 표현을 구성합니다. 공유하는 것은 진행 판정이며, 음성은 별도 서비스가 전달합니다.



1.1 무엇을 공유하고 무엇을 로컬로 두는가

소유자	소유하는 정보	경계를 나눈 이유
게임 서버	참가·역할·준비·정답·문 허가·해금	한 곳에서 일관되게 진행을 승인
각 클라이언트	위치·시점·손·문 각도·화면·소리	엔진의 매 프레임 표현을 서버에서 분리
음성 어댑터·서비스	채널 접속·송수신·장치 상태	게임 판정과 SDK 수명을 별도 관리
콘텐츠 정의	방·퍼즐·앵커·기능의 ID 관계	코드 분기 대신 검증된 데이터로 연결

1.2 요청·응답·로컬 이벤트를 구분한다

입력 → HTTP 요청 → 서버 상태 변경 → 응답 적용 → StateChanged·RoomsChanged 같은 로컬 이벤트 순서입니다. 서버가 클라이언트 객체를 직접 호출하거나, 이 이벤트가 다른 PC로 자동 전파되는 것은 아닙니다. 상태 변경은 조회 응답으로 반영됩니다.

> 현재는 상대 캐릭터 위치와 문 개폐를 복제하는 구조가 아닙니다. 음성 채널 또한 게임 서버가 음성 데이터를 중계한다는 뜻이 아닙니다.

구현 연결: SessionClient · SessionCoordinator · SessionService · SessionVoiceCoordinator · VivoxVoiceGateway

2. 시작 화면에서 탈출하기를 누른다

입장 버튼은 서버를 직접 호출하지 않습니다. 화면 입력과 게임 수명 사이에 조정 계층을 두어 중복 입장과 씬 전환 취소를 다룹니다.

2.1 시작 화면의 기능을 입장 명령으로 바꾼다

플레이 경험	Start_UI의 탈출하기를 누르면 접속을 시작한다. 첫 참가자는 WaitingRoom_UI에서 상대를 기다린다.
처리 흐름	JoinMenuView의 JoinRequested → GameEntryPresenter → GameEntryService.EnterAsync. 접속·데이터 수신 후 최신 참가 인원으로 대기를 결정한다.
화면·소리 결과	이미 두 명이면 별도 대기 화면을 만들지 않고 Ingame 로딩으로 진행한다. 이것은 양쪽 준비를 생략하는 동작이 아니다.

2.2 환경에 맞는 접속 설정을 선택한다

실행 환경	설정 선택	책임
유니티 편집기	editor	개발 환경 선택
개발 빌드	testBuild	테스트 환경 선택
일반 빌드	stableBuild	배포 환경 선택

connection.json을 읽는 UnityClientConnectionConfig가 엔진 환경을 일반 설정 객체로 전달합니다. ClientConnectionSettings는 형식·주소를 검사합니다. 화면은 주소 문자열의 유효성이나 HTTP 생성 방식을 알 필요가 없습니다. 공개 문서에는 실제 운영 주소를 싣지 않습니다.

2.3 씬 이동이 입장 작업을 끊지 않게 한다

입장 초기에는 화면 요청의 취소를 따릅니다. 씬 전환이 시작되면 GameFlowBootstrapper의 게임 소유자 수명으로 넘깁니다. Join 화면이 사라졌다는 이유로 Ingame 생성 작업까지 취소되는 것을 막기 위한 명시적인 전환 경계입니다.

> 선택한 설계: 화면과 조정자의 분리, 명시적 의존성 주입, 상태 변경 관찰. 이득은 UI 교체 시 접속 규칙을 보존하는 것이며, 대가는 취소 토큰과 구독의 소유자를 함께 관리해야 한다는 점입니다.

구현 연결: Session/Presentation/JoinMenuScreen.cs · JoinMenuView.cs · World/GameEntryPresenter.cs · Configuration/ClientConnectionSettings.cs

3. 두 사람의 준비가 끝나면 시작한다

두 명이 접속했다는 사실과, 두 사람이 플레이할 준비를 끝냈다는 사실은 다릅니다.



서버 단계	조건	클라이언트 행동
Waiting	참가자 2명 미만	입장 화면에서 상대 대기
Loading	2명 접속·준비 미완료	각자 방·플레이어·입력 구성
Playing	2명 모두 ready	대기 표시 해제·조작 허용

3.1 준비 보고의 실제 조건

GameFlowBootstrapper.IsGameplayReady는 월드, 활성 플레이어, 이동·시점·상호작용 준비, 활성 Ingame 씬을 함께 확인합니다. SessionSceneLoadedController가 ready를 보고하고, SessionStartGate가 서버의 Playing을 확인해야 조작을 허용합니다. 방만 보인다고 ready를 보내지 않습니다.

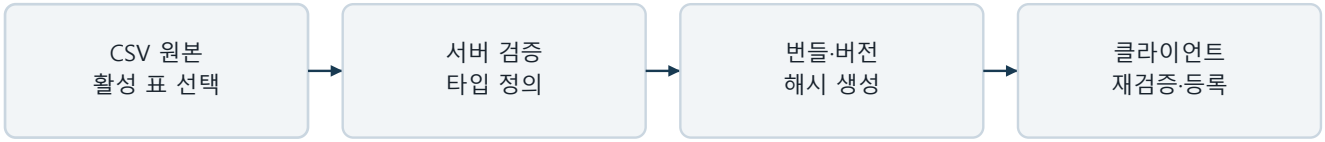
> 그림의 서버→클라이언트 점선은 조회 요청의 응답입니다. 서버 푸시가 아닙니다. 실패 시 이번 입장에서 확보한 참가 자리를 정리하여, 준비되지 않은 사용자가 서버 자리를 점유하지 않게 합니다.

구현 연결: src/models/GameSession.ts · SessionStartGate.cs · SessionSceneLoadedController.cs · SessionGameplayDriver.cs

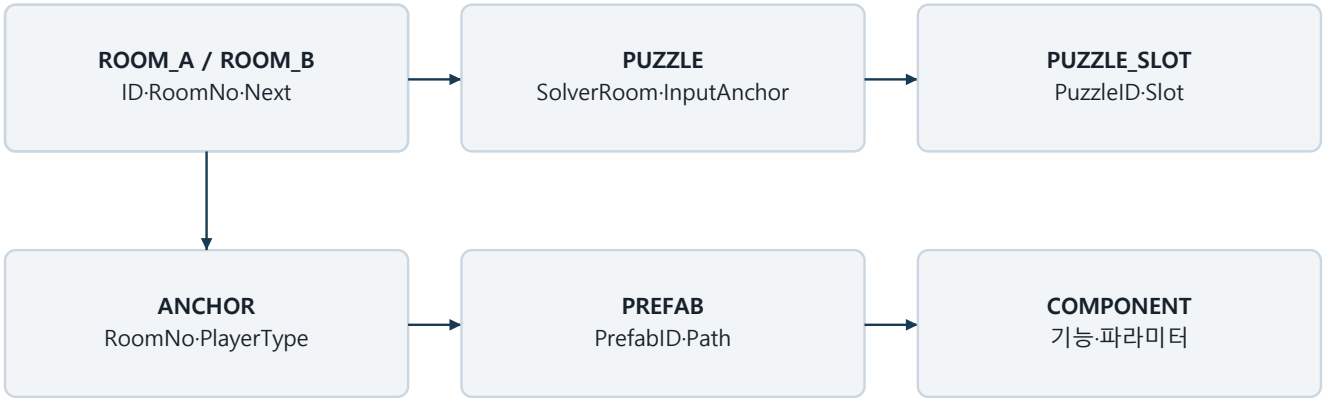
4. 내려받은 데이터가 게임 정의가 된다

서버와 클라이언트가 같은 표를 읽더라도, 원본 파일과 검증된 정의, 플레이 중 바뀌는 상태는 서로 다른 책임입니다.

4.1 서버 원본에서 클라이언트 정의까지



서버는 시작 때 CSV를 읽고 검증합니다. SessionService가 검증된 메모리 원본으로 번들을 만들어 재사용하므로 요청마다 파일을 다시 읽지 않습니다. 번들의 버전은 정렬·직렬화한 표 내용의 SHA-256입니다. 클라이언트는 수신한 번들도 검증한 뒤 새 CsvDataStore에 등록합니다.



REGISTRY가 활성 표를 선택하고, 양쪽 검증기가 관계를 확인

4.2 표 사이를 잇는 키

연결	읽는 의미
ROOM.ID / RoomNo / Next	행 ID·사람이 읽는 방 번호·다음 행 ID를 구분
ANCHOR.RoomNo + PlayerType	방과 A/B/ALL 역할로 배치 선택
ANCHOR.PrefabID → PREFAB	실제 프리팹과 선언 기능을 선택
PUZZLE.SolverRoom / InputAnchor	정답 입력 역할의 방 행과 상호작용 행 연결
PUZZLE_SLOT.PuzzleID / Slot	입력 칸의 순서·표시·판정 정의 연결

> 현재 활성 표는 14개입니다. 필수 값·타입·범위·ID 중복·참조·기능 조합을 검증합니다. 사운드는 GameFeedback 에셋을 사용하고 별도 조명 스폰 CSV는 사용하지 않습니다. 비활성 WIRE·ARROW 등의 확장 정의를 현재 콘텐츠로 설명하지 않습니다.

구현 연결: src/data/GameDataBundle.ts · CsvValidator · CsvGameData · Assets/11.CSV/REGISTRY.csv

5. 역할에 맞는 방과 사물을 만든다

데이터의 ID가 프리팹을 선택하고, 앵커 이름이 저장된 공간의 실제 위치를 찾습니다. 선언과 엔진 객체가 만나는 조립 단계입니다.

5.1 방 계획에서 실제 계층까지

플레이 경험	A와 B는 같은 방 번호에서도 서로 다른 방·소품·단서를 본다.
처리 흐름	RoomLoadPlan이 역할별 ROOM과 ANCHOR를 선택한다. RoomSceneAssembler가 방 프리팹과 위치·삽입 대상을 연결하고 기능 팩토리가 COMPONENT 선언을 실제 구성 요소로 바꾼다.
화면·소리 결과	RoomSceneView가 준비된 방과 앵커를 소유한다. 해금된 다음 방을 추가해도 이전 방은 유지한다.

5.2 두 가지 배치 방식과 책임

방식	행동	의도
저장된 배치	프리팹에 저장한 배치 사용	작업자가 만든 전체 방 연결을 유지
연결점 정렬	출입 연결점으로 다음 방 정렬	연결 규칙을 별도 계산 경계로 분리
앵커 삽입	위치와 프리팹·기능 선언 결합	공통 생성 구조로 소품·조명 구성

앵커는 정확한 이름과 개수까지 맞아야 합니다. CSV 행이 있어도 방 프리팹 안에 위치 기준점이 없으면 로드를 실패시킵니다. 빈 오브젝트를 임의 위치에 만드는 대체 처리는 데이터 오류를 숨길 수 있으므로 정상 경로로 사용하지 않습니다.

5.3 반복 조회는 계획에 묶어 캐시한다

RoomLoadPlan은 한 입장의 정의를 바탕으로 방 ID 사전·프리팹 경로·방별 앵커·진행 순서를 캐시합니다. 재입장하면 새 계획을 사용합니다. Transform을 전역 보관하는 방식과 달리, 씬이 사라진 뒤 파괴된 객체 참조가 남는 범위를 줄입니다.

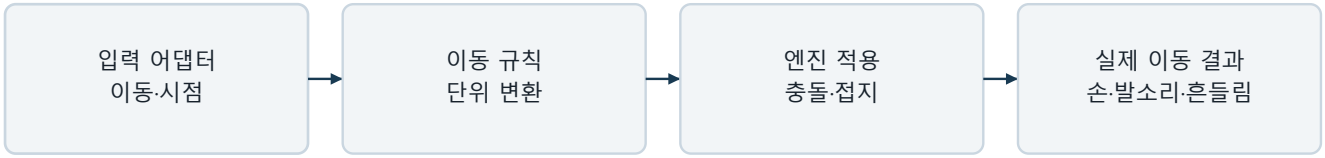
> 선택한 설계: 생성 책임을 모은 팩토리와 데이터 정의 기반 조립. 새 기능이 기존 계약을 벗어나면 해당 기능 구현과 검증도 추가해야 하며, CSV 한 행만으로 임의 스크립트를 실행하는 구조는 아닙니다.

구현 연결: World/RoomLoadPlan.cs · RoomSceneAssembler.cs · RoomSceneView.cs · RoomConnectionAlignment.cs · PrefabFeaturePlan.cs

6. 플레이어가 이동하고 손이 움직인다

플레이어 입력과 실제 이동 결과를 구분합니다. 벽에 막혀 움직이지 않을 때 걷는 소리와 손 동작만 계속되는 문제를 줄이기 위한 기준입니다.

6.1 입력 → 이동 계산 → 충돌 적용 → 표현



InputSystemMoveInput·InputSystemPointerInput이 입력 시스템을 감쌉니다. PlayerMovementMotor와 이동 Driver가 수치·시간·충돌 적용 경계를 나누고, CHARACTER_STAT의 정수 설정을 미터·초 등 엔진 단위로 바꿉니다. 정수 데이터 사용 자체가 엔진 물리의 결정성을 보장하지는 않습니다.

6.2 조작 잠금의 공통 계약

플레이 경험	확대 화면·상대 대기·엔딩 중에는 이동이 잠기고, 화면을 닫아도 다른 잠금이 남아 있으면 움직이지 않는다.
처리 흐름	PlayerControlState가 활성·포커스·일시정지·게임 허용·UI 잠금 수를 합쳐 GameplayEnabled를 계산한다. BlockForUi가 반환한 자원을 해제하면 해당 잠금만 줄인다.
화면·소리 결과	입력·카메라·상호작용이 같은 조작 상태를 따르므로 화면별로 서로 다른 이동 금지 규칙을 복제하지 않는다.

6.3 공통 손 애니메이션과 생성 수명

A/B 손은 같은 제어기의 대기·걷기 동작을 사용합니다. MotionApplied로 받은 접지 이동 거리를 시간으로 나눈 속도가 Float Speed로 전달됩니다. LocalPlayerFactory는 비활성 루트에서 의존성을 조립한 다음 활성화하고, PlayerHandAnimationDriver.ValidateRuntimeBinding에서 실제 초기화·파라미터를 검증합니다.

초기화 전에 네이티브 애니메이터 목록을 검사하면 정상 제어기도 누락으로 오인할 수 있습니다. 생성·활성화·검증을 같은 실패 처리 범위에 두고, 실패하면 입력 자원을 해제하고 루트를 파괴합니다. 생성 성공 전에 입력 소유권을 넘기지 않습니다.

구현 연결: Player/LocalPlayerFactory.cs · PlayerControlState.cs · PlayerMovementMotor.cs · PlayerHandAnimationDriver.cs · PlayerGaitDriver.cs

7. 사물을 바라보고 확대 화면을 연다

상호작용은 카메라가 무엇을 바라보는지에서 시작하지만, 확대 화면은 입력·모델 복제·정답 UI의 수명을 별도로 관리합니다.

7.1 보이는 첫 표면의 유효한 대상만 선택한다

플레이 경험	화면 중앙에서 닿는 사물을 클릭해 관찰하거나 입력 화면을 연다. 벽 뒤의 대상은 선택되지 않는다.
처리 흐름	InteractionTargetFinder가 32칸 고정 버퍼의 RaycastNonAlloc 결과에서 자기 플레이어를 제외한 가장 가까운 충돌 표면을 찾는다. 그 표면의 WorldInteractionTarget을 검사한다.
화면·소리 결과	거리·가용 상태를 통과한 대상에 하이라이트와 상호작용을 적용한다. 버퍼가 가득 차면 잘못된 선택을 피하기 위해 이번 선택을 거절한다.

7.2 관찰과 정답 입력을 분리한다

CloseupController는 확대 상태와 조작 잠금을, CloseupPanelView·CloseupPreview는 UI와 복제 모델·카메라·렌더 텍스처를 다룹니다. 입력 가능한 대상이면 PuzzleInputPlan이 다이얼 또는 순서 입력의 선택지·슬롯을 구성합니다. 일반 관찰 대상과 퍼즐 UI가 같은 책임을 갖지는 않습니다.

객체	소유 책임	정리 경계
상호작용 Driver	현재 대상·화면 연결	플레이어 수명
확대 화면	UI·미리보기·조작 잠금	화면 닫기·씬 종료
정답 Controller	선택값·제출 상태·취소	입력 화면 수명

7.3 화면이 먼저 파괴되는 순서도 처리한다

씬 종료 시 Canvas와 플레이어의 파괴 순서를 고정할 수 없습니다. Dispose는 이미 사라진 Unity 객체를 확인하고 반복 정리에도 견디도록 구성합니다. 늦게 끝난 응답은 취소·수명 검사를 통과해야 화면을 갱신합니다. 닫힌 화면을 다시 켜거나 파괴된 객체에 SetActive를 호출하지 않기 위한 처리입니다.

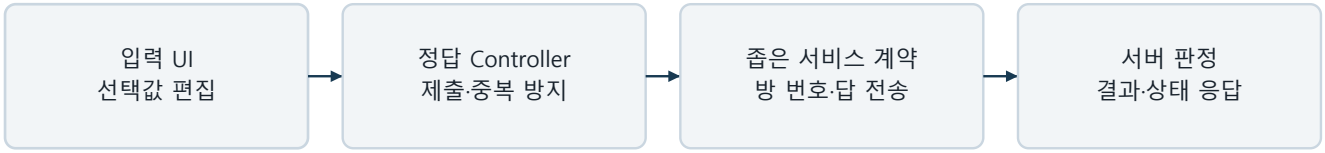
> 선택한 설계: 입력 상태·뷰·엔진 어댑터를 분리하고, 완료된 상태만 관찰합니다. 정적인 확대 모델도 현재 경로에서 반복 렌더링될 수 있으므로 갱신 빈도는 실측 후 개선할 후보입니다.

구현 연결: Interaction/InteractionTargetFinder.cs · CloseupController.cs · CloseupPanelView.cs · CloseupPreview.cs · PuzzleInputPlan.cs

8. 정답을 제출하고 서버가 판정한다

버튼을 눌렀다는 사실은 정답 성공이 아닙니다. 서버가 현재 참가자·진행 상태·입력 대상까지 확인한 응답을 받아야 결과가 됩니다.

8.1 입력 상태와 요청 경로



PuzzleAnswerController는 Editing·Submitting·Correct·Incorrect·Failed·Cancelled 상태를 구분합니다. IRoomAnswerService만 주입받으므로 UI는 참가 토큰이나 HTTP 직렬화를 몰라도 됩니다. 틀린 답과 통신 실패를 같은 화면 상태로 취급하지 않습니다.

8.2 서버가 승인하는 순서

단계	확인·처리	담당
참가 확인	세션 ID·토큰·만료·실제 역할	ParticipantRegistry / SessionService
게임 상태	Playing·현재 역할의 방·해금	SessionService / RoomProgression
퍼즐 판정	입력 개수·순서·대소문자 정규화 비교	PuzzleAnswerRules / SlotAnswerRule
결과 적용	정답 집합·문 허가·해금 갱신	RoomProgression
응답 표현	값·허가·상태 반영 후 완료 알림	RoomInteractionService / UI

8.3 판정 전략을 교체할 수 있게 한 이유

IPuzzleAnswerRule은 판정 규칙의 작은 계약입니다. PuzzleAnswerRules가 퍼즐 Type에 맞는 규칙을 선택합니다. 현재 등록된 종류는 SlotAnswerRule의 순서별 문자열 비교를 공유하며, 각 퍼즐마다 다른 알고리즘이 이미 구현됐다고 설명하지 않습니다. 새 규칙은 등록과 검증·검사를 함께 추가합니다.

> 권한과 비밀은 다릅니다. 판정은 서버가 하지만 현재 활성 CSV 번들에는 정답 정의도 포함됩니다. 이 구조를 정답 은닉·부정행위 방지의 완성으로 볼 수 없습니다. 공개 배포 강화 시 표시용 데이터와 판정 전용 데이터의 분리가 필요합니다.

구현 연결: PuzzleAnswerController.cs · IRoomAnswerService.cs · src/services/SessionService.ts · src/models/PuzzleAnswerRules.ts · RoomProgression.ts

9. 문을 열고 다음 방으로 이동한다

정답 판정, 문 사용 허가, 문짝 회전, 다음 방 생성은 연결되어 있지만 서로 다른 상태입니다.

9.1 허가와 개폐 표현을 따로 소유한다

플레이 경험	정답을 맞히면 진행이 허용된다. 문을 클릭하면 왼쪽 경첩 기준으로 열리고, 이전 방으로 되돌아갈 수도 있다.
처리 흐름	서버가 solved·permitted·unlocked를 별도 집합으로 관리한다. 클라이언트 RoomPermissionService가 허가를 캐시하고 DoorInteractionController가 개폐 상태를 제어한다.
화면·소리 결과	WorldDoorView가 닫힌 위치·회전에서 자세를 다시 계산한다. 문짝과 충돌체가 같이 움직이고, 각도 누적 오차를 줄인다.

9.2 방 번호와 행 ID를 혼동하지 않는다

역할	1번 방 행	Next가 가리키는 행	2번 방
A	1001	1003	RoomNo = 2
B	1002	1004	RoomNo = 2

API에는 사람이 이해하는 roomNo를 보내고, 서버는 인증된 역할의 ROOM에서 행을 찾습니다. 해금 응답은 행 ID 목록입니다. 클라이언트는 이를 RoomLoadPlan으로 해석하고 RoomsChanged를 통해 필요한 방만 추가합니다. 단순히 방 번호에 1을 더하거나 A/B의 행 ID를 공유하지 않습니다.

9.3 왕복과 비동기 실패 처리

문 허가는 입장 수명 안에서 유지하지만 문이 열렸는지는 로컬 상태입니다. 재입장 때 허가 캐시의 세대를 올리고 목록을 비웁니다. 이전 요청의 늦은 응답은 세대가 다르면 무시합니다. 이미 생성한 방과 실패한 생성 시도를 추적하여 조회 때마다 같은 오브젝트를 복제하지 않게 합니다.

> 진행 해금은 공간 이동 감지가 아닙니다. 해금되어 방이 준비돼도 플레이어가 아직 문을 통과하지 않았을 수 있습니다. 배경음은 다음 장에서 설명하는 실제 문 경계 통과를 따릅니다.

구현 연결: RoomPermissionService.cs · DoorInteractionController.cs · WorldDoorView.cs · IngameWorldDriver.cs · src/models/RoomProgression.ts

10. 이동에 맞춰 공간의 소리가 바뀐다

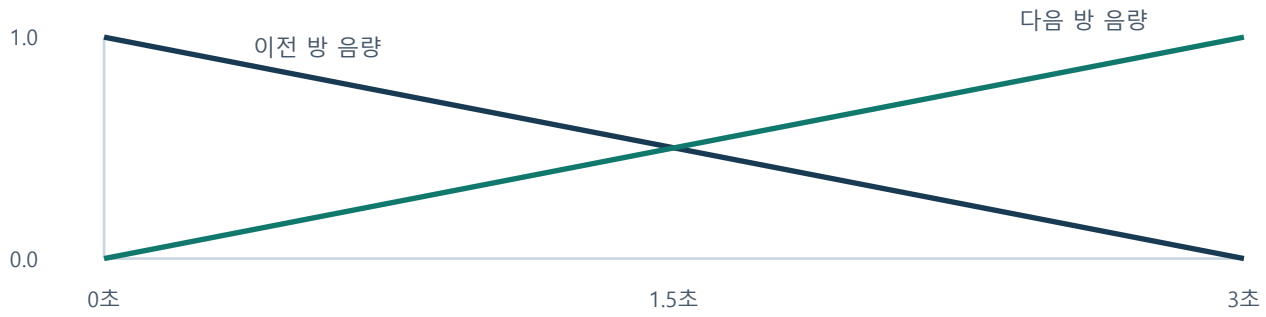
소리의 발동 근거를 실제 이동, 확대 UI, 문 경계로 나눕니다. 하나의 전역 소리 관리자에 모든 조건을 모으지 않습니다.

10.1 보행과 클릭의 서로 다른 근거

소리·표현	발동 근거	현재 정책
발소리·카메라 흔들림	접지한 실제 수평 이동	Silent_Footstep_1 → 2 → 3 순환
손 대기·걷기	충돌 적용 후 이동 속도	A/B 공통 애니메이션
클릭·다이얼·UI음	확대 UI의 의미 있는 입력	일반 모든 UI에 자동 재생하지 않음
방 배경음	열린 문 영역의 실제 통과	다음 방으로 약 3초 전환

10.2 열린 문 영역을 지나는 순간을 사용한다

RoomDoorwayAudio는 문 앵커 좌표계와 닫힌 문 영역으로 경계를 구성합니다. 이전·현재 플레이어 위치가 유효한 문 영역을 가로질렀고 문이 열려 있을 때 방 전환을 선택합니다. 멀리 떨어진 무한 평면의 반대편에 있다는 이유만으로 배경음이 바뀌지 않게 합니다.



10.3 역주행할 때도 현재 음량에서 이어간다

AmbientCrossfade는 모든 채널의 현재 음량을 전환 시작값으로 보관합니다. 전환 중 2번 방에서 다시 1번 방으로 돌아가거나 3번 방을 선택해도, 처음 음량으로 뛰지 않고 현재 값에서 새 목표로 이어갑니다. 전환이 끝난 이전 채널은 정지시킵니다.

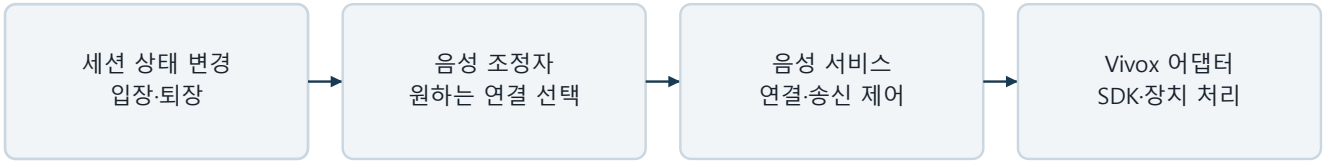
> 음원과 결과 UI는 GameFeedback 에셋의 직접 참조로 구성합니다. SOUND·SOUND_LINK CSV는 현재 사용하지 않습니다. 문 경계 전환과 조명 프로브 적용은 별도 책임입니다.

구현 연결: PlayerGaitDriver.cs · Audio/RoomDoorwayAudio.cs · DoorwayCrossingFrame.cs · AmbientCrossfade.cs · GameFeedbackDriver.cs

11. 다른 방의 상대와 음성으로 협력한다

상대의 단서를 말로 전달하는 경험을 게임 참가 상태에 연결하되, 음성 SDK가 게임 규칙의 권한을 갖지 않게 합니다.

11.1 게임 세션과 음성 연결의 접점



플레이 경험	같은 세션의 두 사용자가 음성 채널에 연결되어 자기 방의 단서를 전달한다.
처리 흐름	SessionVoiceCoordinator가 ISessionConnectionService.StateChanged를 관찰한다. 공개 세션 ID·플레이어 ID·역할을 VoiceConnectionRequest로 바꿔 IVoiceChatService에 전달한다.
화면·소리 결과	VivoxVoiceGateway가 인증·채널·전송을 처리하고, 별도 상태 화면이 연결과 장치 상태를 표시한다.

11.2 연결 변경을 순서대로 처리한다

연결과 끊기가 겹치면 이전 비동기 작업이 늦게 끝날 수 있습니다. 조정자는 원하는 대상과 버전을 보관하고 한 작업 흐름에서 연결 변경을 순서대로 처리합니다. 대상이 바뀌면 먼저 송신을 멈춘 뒤 최신 연결을 반영합니다. 종료 시 구독을 해제하고 연결 정리가 끝날 때까지 추적합니다.

11.3 게임 통신과 음성 통신의 독립성

게임 경로	음성 경로
HTTP / Protobuf 요청·응답	Unity 인증 / Vivox SDK
게임 참가 토큰·준비·정답·해금	음성 채널·로그인·마이크·송수신
게임 규칙과 세션의 권한	협력 대화를 전달하는 외부 기능

> 어댑터 경계를 둬 SDK 의존성을 격리했지만, 실제 음성 품질·권한·마이크 장치·서비스 설정은 두 PC에서 검증해야 합니다. 게임 서버의 health 성공만으로 음성 연결을 확인할 수 없습니다.

구현 연결: Voice/Runtime/SessionVoiceCoordinator.cs · VoiceChatService.cs · IVoiceGateway.cs · Infrastructure/VivoxVoiceGateway.cs

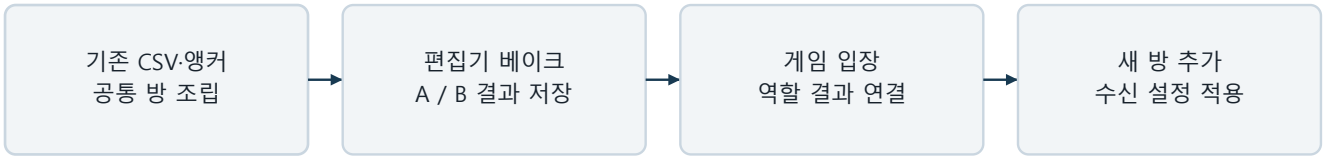
12. 미리 계산한 조명으로 공간을 표현한다

방은 런타임에 추가되지만 빛은 역할별 최종 배치를 기준으로 미리 계산합니다. 실시간 조명 계산을 줄이면서 기존 방 생성 구조를 유지하기 위한 선택입니다.

12.1 앵커 스폰과 조명 계산은 별개다

플레이 경험	서로 연결된 이전·다음 방을 보면서 방마다 의도한 분위기를 경험한다.
처리 흐름	기존 ANCHOR_NONINTERACT → PREFAB으로 조명 위치를 조립한다. 편집기에서 A/B 전체 배치를 구성하고 프로브를 베이크해 각각 저장한다.
화면·소리 결과	실행 중 RoomProbeLightingDriver가 역할에 맞는 결과 하나를 복제·연결하고, 새 방의 메시와 손에 프로브 보간 수신을 설정한다.

12.2 제작과 실행의 비용을 나눈다



런타임에 방을 생성할 때마다 다시 굽지 않습니다. 조명 원본 에셋과 실행 복제본을 구분하고, Unity 전역 프로브 연결과 기존 방향광 상태를 어댑터가 소유합니다. 종료 시 자신이 소유한 상태만 복원하여 새 입장의 조명을 덮어쓰지 않게 합니다.

12.3 프로브를 선택한 범위와 한계

프로브는 위치별 빛을 부드럽게 근사합니다. 정교한 벽 그림자나 선명한 전등 무늬를 만드는 라이트맵과 같은 결과를 보장하지 않습니다. 큰 메시에서는 조명 변화가 균일해 보일 수 있습니다. 현재 목적은 정교한 실시간 그림자보다 방의 기본 조명과 분위기입니다.

> Baked 광원을 생성하는 것만으로 새 빛이 계산되지는 않습니다. 앵커·방·소품·광원 색과 강도를 바꾸면 해당 역할의 결과를 다시 베이크해야 합니다. 결과가 없을 때는 경고하고 기존 조명을 유지합니다.

구현 연결: Editor/RoomProbeBakeTools.cs · Data/RoomProbeLightingSettings.cs · World/RoomProbeLightingDriver.cs · Docs/RoomProbeLighting.md

13. 마지막 방은 두 사람을 기다린다

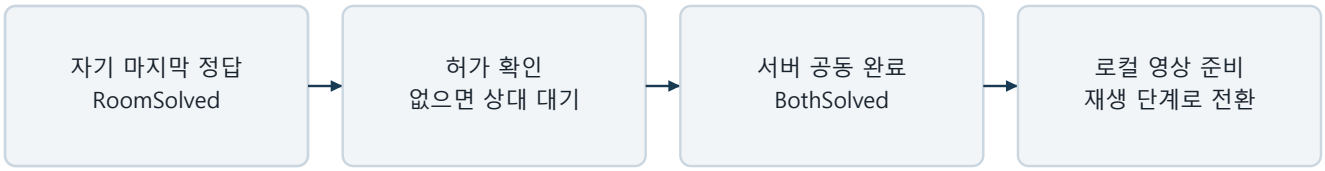
7번 방의 완료 조건은 문을 열거나 출구를 지나는 순간이 아닙니다. 두 사용자의 마지막 정답이 모두 성공한 순간이 기준입니다.

13.1 먼저 맞힌 사용자는 대기한다

상황	서버 판정	클라이언트 상태
한 명만 마지막 정답 성공	정답 true / 공동 문 허가 false	조작 잠금·상대 정답 대기
다른 사용자도 정답 성공	BothSolved 충족	각자 완료 확인 후 영상 준비
대기 중 연결 종료	정상 완료로 대체하지 않음	연결 종료 안내·진행 중단

13.2 마지막 방을 데이터로 식별한다

EndingVideoDirector는 현재 역할의 활성 방 중 Next=0이고 ExitRule=BothSolved인 종단 방 하나를 찾습니다. 현재 데이터에서 방 번호는 7이며 A/B는 서로 다른 마지막 퍼즐을 갖습니다. 코드가 문 오브젝트 이름이나 문 회전 상태로 엔딩을 추정하지 않습니다.



13.3 완료 확인은 게임 상태 조회와 분리한다

EndingCompletionController는 자기 마지막 정답을 맞힌 뒤에만 완료 확인을 시작합니다. 대기 중 약 1초 간격으로 문 허가를 재조회하고, pending으로 중복 요청을 막습니다. 일반 Playing 상태 조회의 기본 5초와 목적이 다릅니다. 두 PC가 같은 프레임에 영상을 시작하도록 동기화하는 장치는 아닙니다.

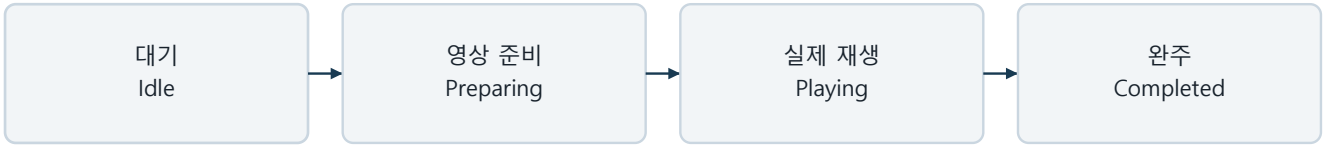
> 선택한 설계: 공동 완료 규칙은 서버·데이터에, 대기와 영상 표현은 클라이언트에 둡니다. RoomSolved는 성공 이후의 로컬 이벤트이며, 다른 PC의 정답을 직접 전달하는 전역 이벤트가 아닙니다.

구현 연결: World/EndingCompletionController.cs · EndingVideoDirector.cs · src/data/RoomExitRules.ts · src/models/RoomProgression.ts

14. 영상이 끝나면 결과와 종료를 보여준다

공동 완료 확인, 영상 준비, 실제 재생 시작, 완주, 결과 표시의 시점을 구분합니다. 탈출 시간은 이 중 실제 재생 시작에 고정합니다.

14.1 영상 재생을 작은 상태 흐름으로 관리한다



EndingPlaybackController는 IEndingVideoPlayback 계약으로 Prepare·Play와 Prepared·Finished·Failed를 사용합니다. 엔진의 VideoPlayer와 렌더 텍스처는 EndingVideoView가 관리합니다. 준비 제한 시간과 실패 상태를 일반 C#으로 검사할 수 있고, 재생 실패는 정상 완주로 취급하지 않습니다.

14.2 탈출 시간의 의미

시점	처리
시작	Ingame 씬 로드 완료 후 기록한 클라이언트 시간
종료	실제 PlaybackStarted 때 한 번 고정
표시	영상 완주 후 Result_UI에 탈출 시간 / 시:분:초
포함·제외	씬 진입 이후 준비·상대 대기 시간 포함 / 영상 재생 시간 제외

EscapeRunClock은 각 클라이언트의 단조 시간을 사용합니다. A/B의 씬 진입·영상 시작 시점이 다를 수 있으므로 같은 기록을 보장하지 않으며 서버 공인 순위 기록도 아닙니다. 서버에서 공정한 기록을 남기려면 측정 기준과 권한을 별도로 설계해야 합니다.

14.3 결과 화면의 자원과 종료

Result_UI를 만들면 왼쪽에 탈출 시간, 오른쪽에 기록을 표시하고 버튼을 게임종료 동작에 연결합니다. 결과 화면 생성 성공 뒤 영상 디코더·렌더 텍스처를 정리합니다. 확인 버튼은 빌드에서 Application.Quit, 편집기에서는 Play 종료로 동작합니다. 현재 종료 입력은 버튼이며 예전 Enter 키 계획과 구분합니다.

> 영상은 직접 참조한 VideoClip을 재생합니다. 월드에서 같은 이름의 모델링을 찾아 실행하지 않습니다. 실패·중단 시에는 이벤트 구독과 영상 자원을 정리하며, 대기 UI와 결과 UI의 수명도 별도로 닫습니다.

구현 연결: EndingPlaybackController.cs · EndingVideoView.cs · EndingVideoDirector.cs · EscapeRunClock.cs · EndingResultView.cs

15. 퇴장과 파괴까지 객체 수명을 닫는다

기능이 성공했을 때만큼 실패·씬 교체·재입장·종료의 순서가 중요합니다. 각 자원은 만든 주체와 해제 책임이 연결되어야 합니다.

15.1 수명 범위별 소유권

범위	주요 소유자	정리하는 것
게임 전체	GameFlow / SessionBootstrapper	서비스·진행 작업·세션 연결
한 번의 접속	SessionClient / Coordinator	취소·연결 버전·자격 상태
플레이어	PlayerSpawnDriver / Factory	입력·이동·상호작용·조명·피드백
확대·결과 화면	각 View / Controller	버튼 구독·복제 모델·조작 잠금
음성·영상	해당 외부 어댑터	SDK 연결·콜백·엔진 자원

15.2 비동기 응답이 새 입장을 덮지 않게 한다

접속 소유자·호출자·연결의 취소 토큰을 묶어 작업 수명을 드러냅니다. SessionClient의 연결 버전과 문 허가 캐시의 세대는 재입장 이전의 늦은 응답을 차단합니다. 취소는 예외를 숨기는 장치가 아니라 더 이상 반영할 수 없는 결과를 구분하는 조건입니다.

15.3 서버에 남는 자리를 회수한다

정상 leave는 일반 요청과 heartbeat의 정리가 끝난 뒤 독점 실행합니다. 서버 응답을 받지 못해도 클라이언트는 로컬 수명을 닫고 확인되지 않은 퇴장을 알립니다. 서버 ParticipantRegistry는 별도의 생존 임대로 만료된 참가자를 제거합니다. 정상 종료와 비정상 연결 단절이 같은 신호에만 의존하지 않습니다.

> 현재 기본값은 heartbeat 약 3초, 서버 참가 만료 10초, 만료 검사 0.5초입니다. 새 접속자가 만료된 빈자리를 확인하도록 join 전에 만료를 정리합니다. 서버 재시작은 메모리 진행을 복구하지 않습니다.

구현 연결: SessionClient.cs · SessionRequestRunner.cs · Player/InputActionLease.cs · ParticipantRegistry.ts · common/liveness/HeartbeatLease.ts

16. 이 흐름을 지탱하는 계층과 패턴

패턴 이름을 많이 붙이는 대신, 책임과 변경 이유가 실제로 다른 지점에서만 경계를 만듭니다.

16.1 데이터·상태·규칙·표현·인프라

계층	클라이언트	서버
데이터 정의	CsvGameData·입력 계획·표현 설정	CsvGameData·CSV 행 모델
가변 상태·규칙	이동 Motor·입력 상태·종료 상태	GameSession·RoomProgression
응용 조정	GameEntryService·SessionClient	SessionService
표현·전송	View·Driver·Protobuf 전송 구현	Controller·HTTP·직렬화
외부 의존성	Unity·Input System·Vivox	파일·시계·스케줄러·로그 출력

16.2 선택한 패턴과 구체적인 대가

설계 기법	적용과 의도	주의할 점
어댑터	입력·전송·음성·영상의 외부 API 격리	계약과 엔진 수명 둘 다 검사
관찰자	상태 적용 후 화면·월드·음성에 통지	구독 순서·해제·재활성화 책임 필요
전략	IPuzzleAnswerRule로 판정 교체	현재 여러 종류가 같은 규칙 공유
팩토리	플레이어·앵커 구성과 생성 실패 정리	소유권 이관 시점이 명확해야 함
파사드	SessionClient가 공개 기능 계약 제공	거대 객체가 되지 않게 내부 위임
템플릿 메서드	생성 API 호출 틀 + SendAsync 구현	생성물과 수동 구현을 구분

생성자·Initialize로 필요한 서비스를 명시적으로 주입합니다. GameSession의 단계 계산과 Controller의 열거형 상태는 상태 모델이지만, 모두 별도 State 객체를 갖는 GoF State 패턴은 아닙니다. Decorator나 이벤트 소싱을 실제 구현 근거 없이 사용했다고 부르지 않습니다.

> 확장 기준: 명령은 요청이고 이벤트는 확정된 결과입니다. 모든 로컬 계산을 전역 이벤트 버스로 보내지 않습니다. 현재 책임 분리는 전송 교체의 출발점이지만 P2P·호스트 이전·다중 서버 동기화를 자동으로 제공하지는 않습니다.

구현 연결: Docs/RefactorConventions.md · Docs/ClientArchitectureReview.md · docs/ArchitectureReview.md · Generated/Protocol/HttpApi.cs

17. 통신 계약·주기·실패를 다루는 경계

게임 요청은 상태를 바꾸는 명령과 상태를 읽는 조회로 나눕니다. 요청이 겹치는 순서와 응답의 형태까지 계약의 일부입니다.

17.1 작은 계약에서 실제 전송까지

IRoomAnswerService → GameEntryService → SessionClient → SessionCoordinator → HttpSessionGateway → SessionApiClient → 생성 GameApiClientBase → ProtobufGameApiClient 순서입니다. 공개 기능·요청 순서·값 변환·HTTP를 분리한 계층이며, 모든 소비자에게 이 전체 연결을 노출하지 않습니다.

기능	경로 요약	의미
입장	POST /sessions/join	자리·역할·토큰 발급
상태·준비	GET /sessions/{id} · POST …/ready	참가·단계 조회 / 준비 보고
데이터	GET …/data	검증된 활성 CSV 번들
정답·문	POST …/rooms/answer · …/rooms/door	서버 판정 / 문 허가 확인
생존·퇴장	POST …/heartbeat · …/leave	임대 갱신 / 자리 반환

17.2 주기와 제한을 목적별로 나눈다

작업	현재 기본 정책
참가·준비 대기 조회	약 1초 간격
진행 중 상태 조회	약 5초 간격
heartbeat	약 3초 간격 / 요청 제한 2초
일반 HTTP 요청	제한 10초
마지막 정답 후 공동 완료 확인	자기 정답 성공 뒤 약 1초 간격

SessionRequestRunner는 일반 요청 하나와 별도 heartbeat 하나를 제한하고, 대기 명령과 독점 leave를 구분합니다. 무제한 요청 대기열이 아닙니다. 전송 결과는 HTTP 상태뿐 아니라 Content-Type·Protobuf 결과 분기·오류 값까지 검사합니다.

> Protobuf는 메시지 형식이며 암호화가 아닙니다. 현재 HTTP 경로·토큰 정책을 완전한 공개 서비스 보안으로 설명하지 않습니다. HTTPS·인증 강화·속도 제한·정답 전용 데이터 분리는 별도 배포 설계 과제입니다.

구현 연결: Networking/ProtobufGameApiClient.cs · ProtobufApiResponseValidator.cs · SessionRequestRunner.cs · protocol/escape/v1 · protocol/http/openapi.json

18. 제작·검증·배포·로그의 자동화

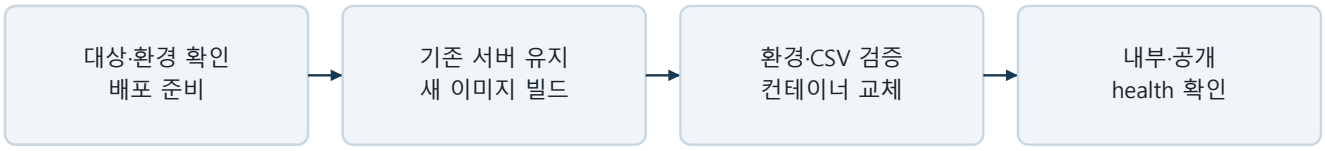
플레이 중 보이지 않는 제작 도구도 같은 계약을 사용합니다. 자동화가 어디까지 해주는지 경계를 명확히 해야 수동 작업 누락을 줄일 수 있습니다.

18.1 세 종류의 원본 계약

원본	생성·검사 결과	자동 작성하지 않는 것
.proto	C#·TypeScript 메시지·직렬화	게임 규칙·프리팹
openapi.json	HTTP 경로·호출 계약·Swagger	실제 서비스 판정 로직
content-contract.json	기능명·인자·퍼즐 종류의 공통 계약	새 기능의 Unity 구현

api:sync는 생성·검사·변경 계획을 수행하고 --apply에서 해시 확인과 백업 후 관리 생성물을 로컬에 반영합니다. GameContentBuilder는 활성 CSV의 PREFAB 참조를 빌드 자원으로 연결합니다. Docker 배포가 로컬 클라이언트 소스나 새 실행 파일을 자동 갱신하지는 않습니다.

18.2 기존 컨테이너를 유지하며 새 이미지를 만든다



기존 배포 도구는 새 이미지의 빌드와 검증이 끝난 뒤 지정 컨테이너를 교체합니다. 교체에는 중단이 있을 수 있고 자동 롤백·무중단 배포를 보장하지 않습니다. 서버 세션이 메모리에 있으므로 교체 시 진행이 초기화됩니다.

18.3 로그는 결과를 관찰하는 별도 출력이다

HttpAccessLogger는 응답 완료·연결 종료를 관찰해 요청 ID·작업·상태 코드·처리 시간·결과를 구조화된 표준 출력으로 남깁니다. 토큰·정답 원문은 로그에 저장하지 않습니다. 출력 실패가 이미 끝난 게임 요청을 실패로 바꾸지 않게 처리합니다.

> Docker의 json-file 회전 기본값은 파일당 10MB·최대 3개입니다. 앱의 오류 전용 파일 저장과는 다른 기능이며, 현재 배포 도구에 CPU·메모리 제한 옵션은 없습니다. Swagger는 /swagger/에서 계약을 보여주며 서버가 실행 중이어야 합니다.

구현 연결: scripts/sync-game-api.cjs · scripts/docker-deploy · Editor/GameContentBuilder.cs · HttpAccessLogger.ts · docs/docker-deployment.md

19. 최적화와 현재 확인 범위

줄인 비용과 남아 있는 비용을 함께 설명합니다. 파일 크기나 순수 코드 검사 결과를 전체 플레이 성능으로 바꾸어 말하지 않습니다.

19.1 실제 적용한 비용 절감

영역	현재 처리	남는 비용·주의
데이터·방 조회	입장별 계획·사전·조회 결과 캐시	입장 초기 구성과 실제 Instantiate
대상 감지	32칸 RaycastNonAlloc 버퍼 재사용	조밀한 충돌 배치에서는 선택 거절 가능
텍스처	일반 BC7·노멀 BC5 적용	손실 압축·기호 판독성 별도 확인
조명	역할별 프로브 베이크·실행 연결	렌더링 보간·큰 메시의 품질 한계
수명·미디어	구독 해제·결과 뒤 영상 자원 정리	실제 장치·종료 순서 통합 검사 필요

2026-10-21 기록에서는 텍스처 형식 조정 후 Windows BuildReport 출력이 약 4.56GB에서 2.16GB로 줄었습니다. 과거 동일 비교 조건의 결과이며 현재 ZIP 크기·현재 전체 메모리·프레임 향상 수치가 아닙니다. 메시 최적화는 별도 작업입니다.

19.2 무엇을 근거로 설명했는가

근거	이번 문서에서의 의미
현재 코드·CSV·설정	호출·권한·연결·수명과 데이터 조건 확인
기존 검사 기록	서버 176개 회귀 등 날짜가 있는 과거 결과
10월 26일 진단 실행	손 초기화 수정안의 A/B 준비·실패 정리 확인
이번 문서 작업	새 전체 빌드·클라우드·두 PC 플레이 검사를 수행한 것은 아님

19.3 공개 범위의 한계와 다음 검증

현재는 프로세스당 메모리 세션 하나입니다. 서버 재시작 복구·다중 매칭·영구 기록·상대 위치 동기화는 구현 범위가 아닙니다. 배포판에서는 양쪽 ready, 7개 방 연결·왕복, 협동 엔딩, 음성·영상·텍스처 판독성을 실제 두 PC로 확인해야 합니다. GPU·메모리 절감은 프로파일링으로 확인합니다.

구현 연결: Docs/TextureCompression.md · RoomProbeLighting.md · 빌드_입장_중단_원인과_수정.md · 양쪽 문서_현행화_점검표.md

20. 코드와 기존 문서 찾아보기

기존 서버·클라이언트 문서를 기능별로 묶었습니다. 이 문서는 플레이 관점의 통합 안내이며, 상세 작업 절차는 각 저장소의 문서를 따릅니다.

20.1 플레이 단계별 코드 색인

단계	클라이언트 경로: Assets/3.Script 아래	서버 경로: src 아래
입장·준비	World/GameEntryService.cs World/SessionStartGate.cs	services/SessionService.ts models/GameSession.ts
데이터·방 생성	World/RoomLoadPlan.cs World/RoomSceneAssembler.cs	data/GameDataBundle.ts data/validation/
플레이어·상호작용	Player/LocalPlayerFactory.cs Interaction/PuzzleAnswerController.cs	models/PuzzleAnswerRules.ts models/RoomProgression.ts
진행·엔딩	World/RoomPermissionService.cs World/EndingVideoDirector.cs	data/RoomExitRules.ts models/RoomProgression.ts
통신·운영	Networking/ProtobufGameApiClient.cs Session/SessionRequestRunner.cs	controllers/SessionController.ts infrastructure/http/HttpAccessLogger.ts

20.2 기존 문서의 핵심을 어디에 반영했는가

기존 문서군	통합 문서 반영
클라이언트 Docs/ClientCodeArchitecture.md 서버 docs/ServerCodeArchitecture.md	전체 흐름·계층·호출 연결: 0~9장, 16~17장
JoinMenu / EntryConsistency / SessionPolling	입장·준비·수명·주기: 2~3장, 15·17장
DataDrivenInteraction / DesignerDataTableGuide	표 관계·기능 선언·입력: 4~8장
GameFeedback / EndingVideo / RoomProbeLighting	소리·음성·조명·결과: 10~14장
RefactorConventions / ArchitectureReview	패턴 선택·확장 경계: 16장
docker-deployment / swagger-api-guide	생성 계약·배포·로그: 18장
TextureCompression / 문서_현행화_점검표	측정 범위·검사 한계: 19장

> 경로는 저장소 안의 상대 경로입니다. 줄 번호는 이후 수정으로 변할 수 있어 고정하지 않았습니다. 문서 내용은 현재 로컬 코드 기준이며, 원격 저장소에 모두 반영됐다는 뜻은 아닙니다.

21. 공개 범위와 저장소

시작 화면의 한 번의 입력이 공동 탈출로 이어지는 경로를 데이터·명령·상태·표현의 연결로 정리했습니다.

21.1 구현을 읽는 네 가지 기준

기준	확인할 질문
권한	누가 참가·준비·정답·문 허가를 확정하는가?
데이터	어느 안정적인 ID가 방·앵커·프리팹·퍼즐을 연결하는가?
순서	명령 성공 뒤 어떤 상태와 완료 이벤트가 적용되는가?
수명	씬 교체·취소·실패 때 누가 구독과 자원을 해제하는가?

21.2 다음 구조 확장에 필요한 일

다중 매칭은 세션 저장소와 세션별 자원 격리를, 영구 진행은 저장·복구 계약을 추가해야 합니다. P2P나 호스트 방식은 판정 주체·상태 복제·명령 순서·시간 기준을 다시 정해야 합니다. 현재의 직렬화 ID와 어댑터는 이를 위한 경계이지만 구현 완료를 의미하지 않습니다.

외부 서비스로 확장할 때는 HTTPS·인증·요청 제한·정답 전용 데이터 분리, 장애 복구·운영 관측을 추가로 설계합니다. 현재 구현과 앞으로 필요한 작업을 구분하고, 실제 플레이 검증 결과를 해당 버전의 코드·설정과 함께 관리합니다.

21.3 Git 저장소

Unity 클라이언트
https://github.com/lkhwcf-arch/-listen

게임 서버
https://github.com/needAction/TSC_REST

주소 근거: 각 프로젝트의 Git origin 설정. 이 문서 작성 과정에서 원격 저장소의 공개 여부·접근 권한·최신 반영 상태를 별도 확인하지 않았습니다.

> 문서 기준일: 2026-10-26. 현재 로컬 구현과 기존 문서의 핵심을 통합했습니다. 원본 형식 참고: 제공된 REC 기술문서의 경험→처리→결과→설계 의도 구성. 게임 내용과 코드 근거는 SILENT 프로젝트에서 다시 확인했습니다.